МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

ОТЧЁТ

по практическому занятию № 2

по дисциплине «Технологии индустриального программирования»

Тема: «Выбор темы проекта. Сбор требований.
Написание документации проекта. UML-диаграммы»

**Спецификация требований к программному обеспечению (SRS)
проекта «Redis Lite – In-Memory Key-Value хранилище
с транзакциями»**

Выполнили студенты группы ЭФБО-04-25 Рожко А.В.
Юрченко Г.В.

Принял преподаватель Афонин С.А.

Москва 2026

СОДЕРЖАНИЕ

1 Введение 3
1.1 Цель 3
1.2 Область действия (Scope) 3
1.3 Определения и аббревиатуры 3
1.4 Команда проекта 4
2 Общее описание 5
2.1 Описание продукта 5
2.2 Функции продукта 5
2.3 Характеристики пользователей 6
3 Детальные требования 7
3.1 Методология сбора требований 7
3.2 Бизнес-требования 8
3.3 Пользовательские требования 8
3.4 Функциональные требования 9
3.5 Нефункциональные требования 11
4 Диаграммы UML 12
4.1 Диаграмма вариантов использования 12
4.2 Диаграмма классов 13
4.3 Диаграмма последовательности 15
Заключение 17
Список использованных источников 18

1 Введение

1.1 Цель

Настоящий документ – спецификация требований к программному обеспечению (Software Requirements Specification, SRS) учебного проекта «Redis Lite». В нём описано, что должна делать система, для кого она предназначена и какие требования к ней предъявляются. Структура документа соответствует рекомендациям IEEE 830 [2], требования разделены на уровни по К. Вигерсу [1]. Документ служит основой для реализации и проверки проекта.

Тема выбрана из перечня тем дисциплины – № 10 «In-Memory Key-Value хранилище с транзакциями (как Redis Lite)». Нам интересно разобраться, как такие хранилища устроены изнутри: работа с сокетами, многопоточность и транзакции.

1.2 Область действия (Scope)

Redis Lite – сетевой сервер на C++, который хранит пары «ключ–значение» в оперативной памяти и позволяет нескольким клиентам по протоколу TCP выполнять команды SET, GET и DEL, а также объединять изменения в транзакции (BEGIN, COMMIT, ROLLBACK).

В рамки проекта входят: сервер, базовые команды, транзакции, одновременная работа нескольких клиентов, время жизни ключей и сохранение данных в файл.

В рамки проекта не входят: репликация, сложные типы данных (списки, хеши), авторизация, графический интерфейс.

1.3 Определения и аббревиатуры

Таблица 1 – Определения и аббревиатуры

Термин	Определение
Хранилище «ключ–значение»	Хранилище, в котором каждая запись – пара из уникального ключа и значения; значение получают по ключу.

Термин	Определение
In-Memory	Хранение всех данных в оперативной памяти, за счёт чего операции выполняются очень быстро.
Транзакция	Группа изменений, которая применяется целиком или не применяется вовсе.
BEGIN / COMMIT / ROLLBACK	Команды начала, фиксации (применения) и отката (отмены) транзакции.
TTL (Time To Live)	Время жизни ключа, по истечении которого ключ считается удалённым.
Клиент	Программа или человек, который подключается к серверу и отправляет команды.
Сокет	Программный интерфейс операционной системы для обмена данными по сети.
TCP	Сетевой протокол с гарантированной доставкой данных.
Мьютекс	Объект синхронизации, который не даёт нескольким потокам одновременно менять общие данные.
SRS	Software Requirements Specification – спецификация требований к ПО.
UML	Unified Modeling Language – язык визуального моделирования программных систем.
MoSCoW	Способ приоритизации: Must (обязательно), Should (желательно), Could (можно), Won't (не в этой версии).

1.4 Команда проекта

Проект выполняют два студента (таблица 2).

Таблица 2 – Распределение работ

Участник	Роль	Что делает
Рожко Александр	Аналитик, разработчик	Сбор требований и SRS; хранилище (Storage), транзакции (Transaction), сохранение в файл; модульные тесты.
Юрченко Георгий	Разработчик	UML-диаграммы; сервер и работа с клиентами (Server, ClientSession); разбор и выполнение команд (CommandParser, Command).

Код хранится в репозитории на GitHub; каждое изменение проверяет второй участник через pull request.

2 Общее описание

2.1 Описание продукта

Redis Lite – упрощённый учебный аналог хранилища Redis [3]. Сервер запускается из терминала и ждёт подключений на порту 6380. Для каждого подключившегося клиента создаётся отдельный поток; все потоки работают с общим хранилищем в памяти, доступ к которому защищён мьютексом.

Команды передаются текстом, по одной в строке, поэтому с сервером можно работать через стандартную утилиту nc (netcat). Пример сеанса:

```
$ nc localhost 6380
SET name Alice
OK
GET name
Alice
BEGIN
OK
SET balance 100
OK
COMMIT
OK
GET city
(nil)
```

Транзакция работает так: после BEGIN изменения клиента не сразу попадают в хранилище, а накапливаются в его личном буфере. По команде COMMIT весь буфер применяется к хранилищу за один раз, по ROLLBACK – просто отбрасывается.

2.2 Функции продукта

– хранение строковых значений по ключам в оперативной памяти;

– команды SET, GET, DEL и EXISTS;

– транзакции BEGIN, COMMIT и ROLLBACK;

– одновременная работа нескольких клиентов;

– время жизни ключей (EXPIRE, TTL);

– сохранение данных в файл и загрузка при запуске (SAVE).

2.3 Характеристики пользователей

Пользователи системы перечислены в таблице 3. Все они умеют работать с командной строкой, поэтому графический интерфейс не нужен.

Таблица 3 – Классы пользователей

Класс	Описание	Что использует
Разработчик	Программист, который подключает хранилище к своей программе, например как кэш. Знает основы работы с сетью.	Команды работы с данными, транзакции, TTL
Пользователь консоли	Студент или тестировщик, который проверяет работу сервера вручную через nc или telnet.	Все команды вручную
Администратор	Тот, кто запускает и останавливает сервер на своём компьютере.	Запуск с параметрами, SAVE, остановка

На диаграмме вариантов использования разработчик и пользователь консоли объединены в одного актора – «Клиент».

3 Детальные требования

3.1 Методология сбора требований

Внешнего заказчика у проекта нет: исходные требования задаёт формулировка темы, остальное мы собирали сами. Использовались следующие методы.

1. **Анализ документов.** Изучили формулировку темы и ограничения дисциплины: проект пишется на C++, библиотеки не должны подменять реализацию, обязательны команды GET, SET, DEL, BEGIN и COMMIT.

2. **Анализ аналогов.** Посмотрели документацию Redis [3]: какие в нём есть команды, как выглядят ответы, как работают время жизни ключей и сохранение на диск. Из Redis взяли названия команд и вид ответов. В Redis нет отката транзакций, а у нас есть ROLLBACK – этого требует тема.

3. **Интервью.** Поговорили с тремя одногруппниками, которые пишут backend-проекты, о том, зачем им могло бы понадобиться такое хранилище. Главное: его используют как кэш и для хранения сессий, поэтому нужно автоматическое удаление старых данных (TTL), а данные не должны пропадать при перезапуске сервера.

4. **Пользовательские истории (User Stories).** Записали потребности в формате «Как <роль>, я хочу <действие>, чтобы <цель>» с критериями приёмки (подраздел 3.3).

5. **Приоритизация MoSCoW.** Разделили требования по важности (таблица 4).

При сборе следили, чтобы в требования попадали потребности, а не готовые технические решения. Например, пожелание «сделайте как в Redis, на epoll» записано как требование «сервер обслуживает несколько клиентов одновременно», а способ реализации (отдельный поток на клиента) выбран уже при проектировании. Так же пожелание «храните данные в JSON» превратилось в требование «данные не пропадают при перезапуске», а формат файла выбрали сами.

Таблица 4 – Приоритеты требований

Категория	Что входит
Must	Сервер на сокетах, несколько клиентов одновременно, команды SET, GET, DEL, QUIT, транзакции BEGIN, COMMIT, ROLLBACK.
Should	EXISTS, время жизни ключей (EXPIRE, TTL), сохранение в файл (SAVE) и загрузка при запуске.
Could	Собственный консольный клиент, команда INFO со статистикой сервера.
Won't	Репликация, сложные типы данных, авторизация, графический интерфейс.

3.2 Бизнес-требования

Бизнес-требования отвечают на вопрос, зачем нужен проект (таблица 5).

Таблица 5 – Бизнес-требования

ID	Бизнес-требование
БТ-1	Разработать к концу семестра (декабрь 2026 г.) работающий сервер хранилища «ключ–значение» с транзакциями и продемонстрировать его на защите.
БТ-2	Получить хранилище, которое можно использовать как быстрый кэш в учебных backend-проектах.
БТ-3	Освоить командную разработку по промышленному процессу: требования, UML, Git, code review.

Проект считается успешным, если все требования с приоритетом Must реализованы и показаны на защите.

3.3 Пользовательские требования

Пользовательские требования записаны в виде пользовательских историй с критериями приёмки (таблица 6).

Таблица 6 – Пользовательские требования

ID	Пользовательская история	Критерий приёмки	Приоритет
ПТ-1	Как разработчик, я хочу сохранить значение по ключу, чтобы потом быстро его получить.	После `SET k v` команда `GET k` возвращает `v`.	Must
ПТ-2	Как разработчик, я хочу получить значение по ключу.	Для несуществующего ключа возвращается (nil), а не ошибка.	Must

ID	Пользовательская история	Критерий приёмки	Приоритет
ПТ-3	Как разработчик, я хочу удалять ненужные ключи.	После `DEL k` команда `GET k` возвращает (nil).	Must
ПТ-4	Как разработчик, я хочу, чтобы несколько программ работали с сервером одновременно.	Два клиента выполняют команды одновременно; отключение одного не мешает другому.	Must
ПТ-5	Как разработчик, я хочу выполнять несколько изменений как одно целое, чтобы данные не остались изменёнными наполовину.	Изменения после BEGIN видны другим клиентам только после COMMIT, причём все сразу.	Must
ПТ-6	Как разработчик, я хочу отменить начатую транзакцию.	После ROLLBACK данные такие же, как до BEGIN.	Must
ПТ-7	Как разработчик, я хочу задавать ключу время жизни, чтобы устаревшие данные удалялись сами.	После истечения срока GET возвращает (nil).	Should
ПТ-8	Как администратор, я хочу, чтобы данные можно было сохранить в файл и они восстанавливались после перезапуска.	После SAVE и перезапуска сервера ключи на месте.	Should
ПТ-9	Как администратор, я хочу запускать сервер на нужном порту.	Порт задаётся параметром при запуске.	Must

3.4 Функциональные требования

Функциональные требования описывают, что именно должна делать система (таблица 7).

Таблица 7 – Функциональные требования

ID	Требование	Приоритет
Сетевое взаимодействие		
ФТ-1	Сервер принимает TCP-подключения на порту, указанном при запуске (по умолчанию 6380).	Must
ФТ-2	Сервер обслуживает несколько клиентов одновременно, каждого в отдельном потоке.	Must
ФТ-3	Команды передаются текстовыми строками: имя команды и аргументы через пробел. Имя команды не зависит от регистра.	Must
ФТ-4	На неизвестную команду или неверное число аргументов сервер отвечает «ERR <причина>» и продолжает работу.	Must
ФТ-5	Команда QUIT закрывает соединение с клиентом.	Must
Работа с данными		

ID	Требование	Приоритет
ФТ-6	`SET key value` сохраняет значение (старое значение перезаписывается) и отвечает OK.	Must
ФТ-7	`GET key` возвращает значение или (nil), если ключа нет.	Must
ФТ-8	`DEL key` удаляет ключ и возвращает 1, если ключ был, иначе 0.	Must
ФТ-9	`EXISTS key` возвращает 1, если ключ есть, иначе 0.	Should
Транзакции		
ФТ-10	`BEGIN` начинает транзакцию. Повторный BEGIN внутри транзакции возвращает ошибку.	Must
ФТ-11	После BEGIN команды SET и DEL не меняют общее хранилище, а записываются в буфер транзакции клиента.	Must
ФТ-12	GET внутри транзакции учитывает изменения из её буфера.	Must
ФТ-13	Другие клиенты не видят изменений транзакции до COMMIT.	Must
ФТ-14	`COMMIT` применяет все изменения из буфера к хранилищу за одну операцию и отвечает OK.	Must
ФТ-15	`ROLLBACK` очищает буфер, не меняя хранилище, и отвечает OK.	Must
ФТ-16	COMMIT или ROLLBACK без BEGIN возвращают ошибку.	Must
ФТ-17	Если клиент отключился посреди транзакции, она отменяется.	Must
Время жизни ключей		
ФТ-18	`EXPIRE key seconds` задаёт время жизни ключа; после его истечения ключ считается удалённым.	Should
ФТ-19	`TTL key` возвращает оставшееся время жизни в секундах: -1, если срок не задан, и -2, если ключа нет.	Should
Сохранение данных		
ФТ-20	`SAVE` сохраняет все данные в текстовый файл dump.txt.	Should
ФТ-21	При запуске сервер загружает данные из dump.txt, если файл существует.	Should

Если две транзакции изменяют один и тот же ключ, итоговым будет значение той, что зафиксирована последней; обнаружение конфликтов между транзакциями в проект не входит.

Сводка команд сервера приведена в таблице 8.

Таблица 8 – Команды сервера

Команда	Пример	Ответ
`SET key value`	`SET name Alice`	OK
`GET key`	`GET name`	Alice или (nil)
`DEL key`	`DEL name`	1 или 0
`EXISTS key`	`EXISTS name`	1 или 0

Команда	Пример	Ответ
`EXPIRE key seconds`	`EXPIRE name 60`	1 или 0
`TTL key`	`TTL name`	Секунды, -1 или -2
`BEGIN`	`BEGIN`	OK
`COMMIT`	`COMMIT`	OK
`ROLLBACK`	`ROLLBACK`	OK
`SAVE`	`SAVE`	OK
`QUIT`	`QUIT`	Соединение закрывается

При ошибке сервер отвечает строкой вида «ERR unknown command».

3.5 Нефункциональные требования

Нефункциональные требования описывают, каким должен быть продукт: насколько быстрым, надёжным и удобным (таблица 9).

Таблица 9 – Нефункциональные требования

ID	Требование
Производительность	
НФТ-1	Команды GET и SET выполняются в среднем быстрее 1 мс, когда клиент и сервер работают на одном компьютере.
НФТ-2	Сервер выдерживает не менее 50 одновременных подключений.
Надёжность	
НФТ-3	Сервер не завершается аварийно при некорректных командах и при внезапном отключении клиента.
НФТ-4	Транзакция применяется атомарно: другие клиенты никогда не видят её применённой наполовину.
НФТ-5	Доступ потоков к общему хранилищу синхронизирован мьютексом (std::mutex), одновременная запись не портит данные.
Удобство использования	
НФТ-6	С сервером можно работать через стандартные утилиты nc или telnet; ответы и сообщения об ошибках понятны человеку.
Ограничения и переносимость	
НФТ-7	Язык – C++17. Используются только стандартная библиотека и POSIX-сокеты; сторонние библиотеки не подменяют реализацию.
НФТ-8	Проект собирается через CMake и работает на Linux и macOS.
Сопровождаемость	
НФТ-9	Код разбит на классы по ответственности: сеть, разбор команд, хранилище, транзакции.
НФТ-10	Классы хранилища и транзакций покрыты модульными тестами (GoogleTest).

4 Диаграммы UML

Модели системы построены на языке UML [4]: диаграмма вариантов использования показывает, кто и что делает с системой, диаграмма классов – из каких частей состоит сервер, диаграмма последовательности – как эти части взаимодействуют при выполнении транзакции.

4.1 Диаграмма вариантов использования

Диаграмма вариантов использования приведена на рисунке 1. Прямоугольник – граница системы, овалы внутри – варианты использования (функции), человечки снаружи – акторы.

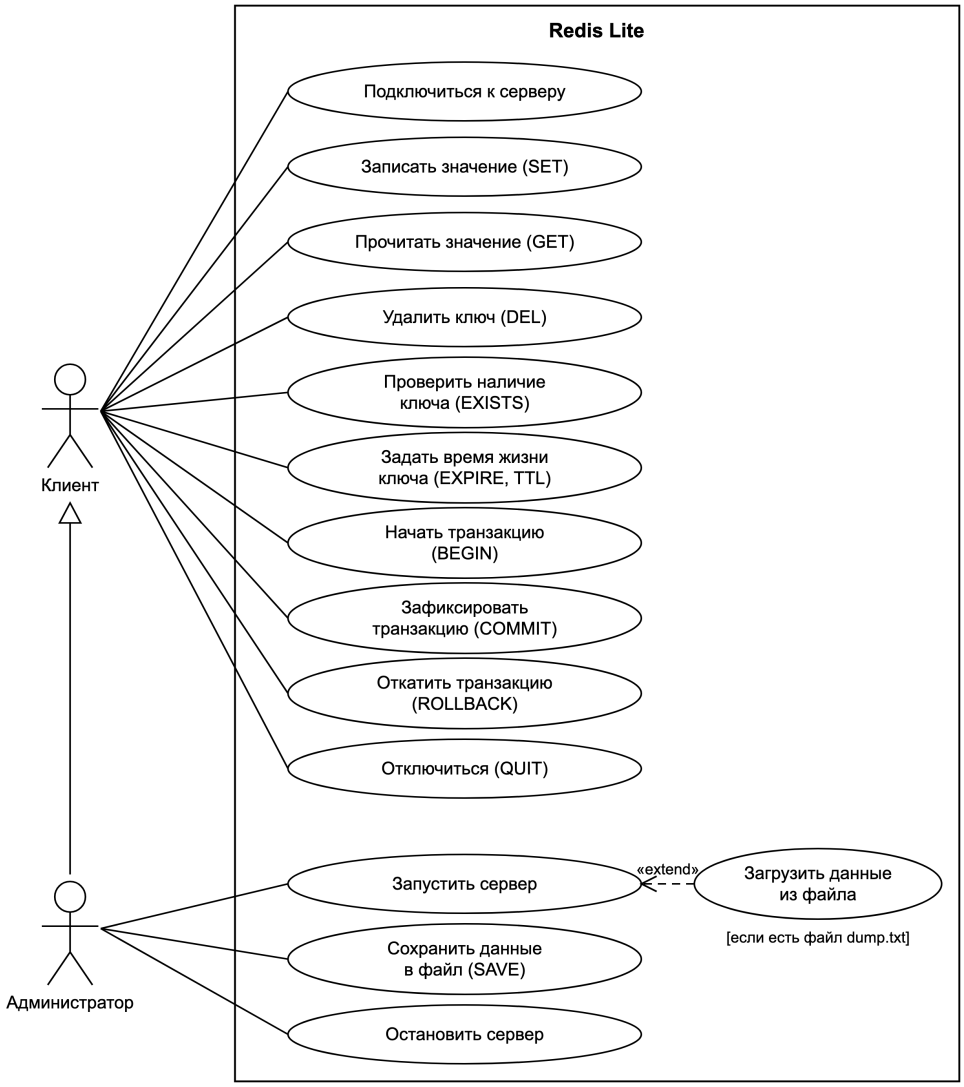


Рисунок 1 – Диаграмма вариантов использования

– **Клиент** работает с данными (SET, GET, DEL, EXISTS, EXPIRE, TTL) и транзакциями (BEGIN, COMMIT, ROLLBACK);

– **Администратор** запускает и останавливает сервер и сохраняет данные в файл. Стрелка обобщения от администратора к клиенту означает, что администратор может делать всё то же, что и клиент.

Отношение «extend» показывает необязательное расширение: при запуске сервера данные загружаются из файла, только если файл dump.txt существует.

4.2 Диаграмма классов

Диаграмма классов приведена на рисунке 2, назначение классов – в таблице 10. Кроме показанных, у класса Command есть наследники для остальных команд (ExistsCommand, ExpireCommand, TtlCommand, SaveCommand, QuitCommand); они устроены так же и на диаграмме не показаны.

Таблица 10 – Классы системы

Класс	Назначение	Кто делает
Server	Принимает подключения и для каждого клиента создаёт сессию в отдельном потоке	Юрченко Г.В.
ClientSession	Обслуживает одного клиента: читает строки, выполняет команды, отправляет ответы; хранит открытую транзакцию	Юрченко Г.В.
CommandParser	Разбивает строку на слова и создаёт объект нужной команды	Юрченко Г.В.
Command и наследники	Абстрактная команда и её реализации для каждой команды протокола	Юрченко Г.В.
Storage	Общее хранилище: таблица «ключ → запись», защищённая мьютексом; сохранение в файл	Рожко А.В.
Entry	Запись хранилища: значение и момент истечения срока жизни	Рожко А.В.
Transaction	Буфер изменений транзакции: ключ → новое значение или отметка об удалении	Рожко А.В.

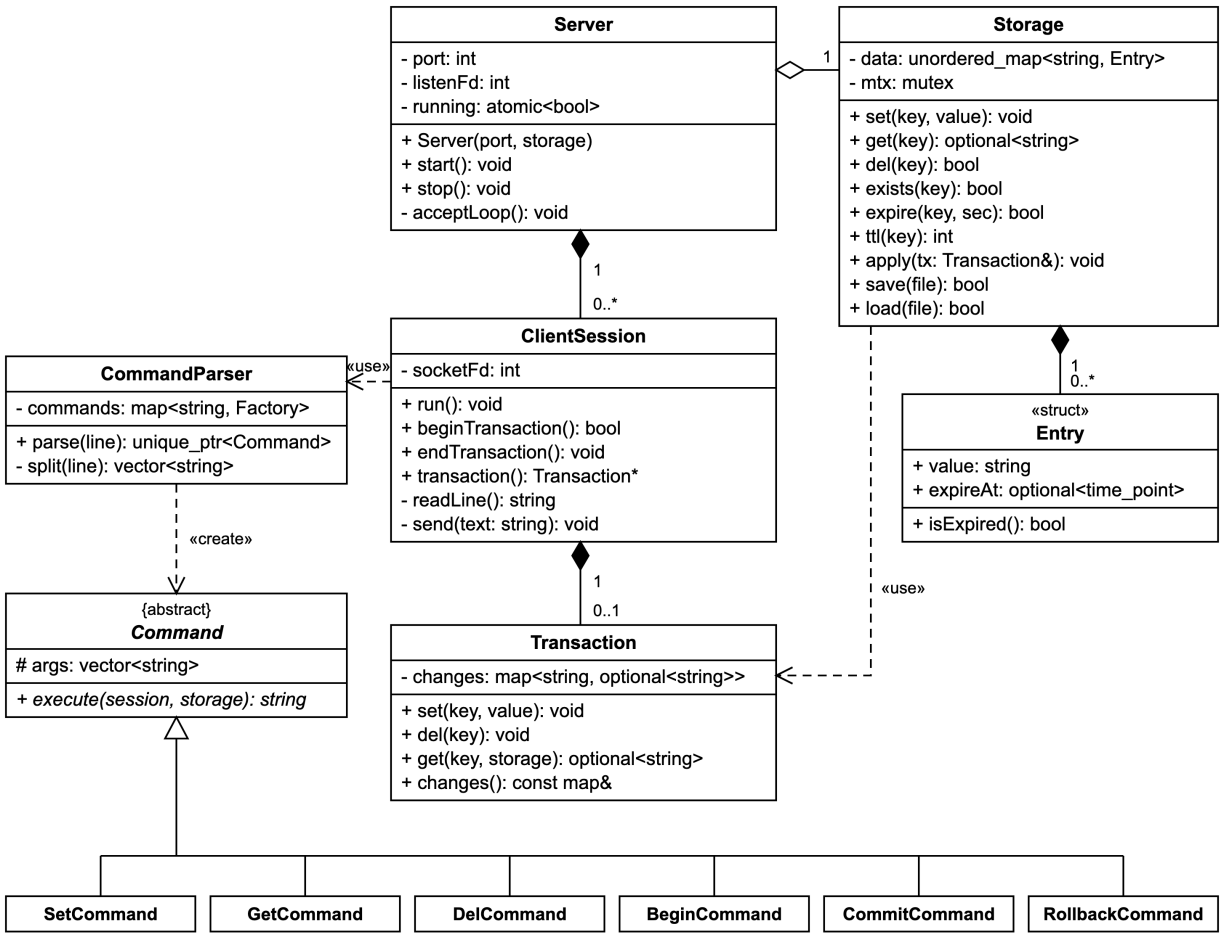


Рисунок 2 – Диаграмма классов

На диаграмме использованы отношения:

– **композиция** (закрашенный ромб): сервер владеет сессиями, сессия – открытой транзакцией, хранилище – записями; части не существуют без целого;

– **агрегация** (пустой ромб): сервер использует хранилище, но не владеет им – хранилище создаётся в функции main и передаётся серверу;

– **наследование** (стрелка с треугольником): конкретные команды наследуют абстрактный класс Command и реализуют метод execute;

– **зависимость** (пунктирная стрелка): сессия использует CommandParser, CommandParser создаёт команды, Storage принимает транзакцию в методе apply.

4.3 Диаграмма последовательности

Диаграмма последовательности (рисунок 3) показывает основной сценарий – выполнение транзакции:

1. Клиент отправляет «BEGIN». Сессия с помощью CommandParser получает объект команды и вызывает execute(); команда создаёт объект Transaction.

2. Каждая команда SET или DEL (фрагмент loop) записывает изменение в буфер транзакции, а не в хранилище, поэтому другие клиенты его не видят.

3. Фрагмент alt показывает два варианта. При COMMIT команда вызывает Storage::apply(tx): хранилище захватывает мьютекс и применяет все изменения разом. При ROLLBACK хранилище не трогается.

4. В обоих случаях сессия удаляет объект Transaction и отвечает клиенту «OK».

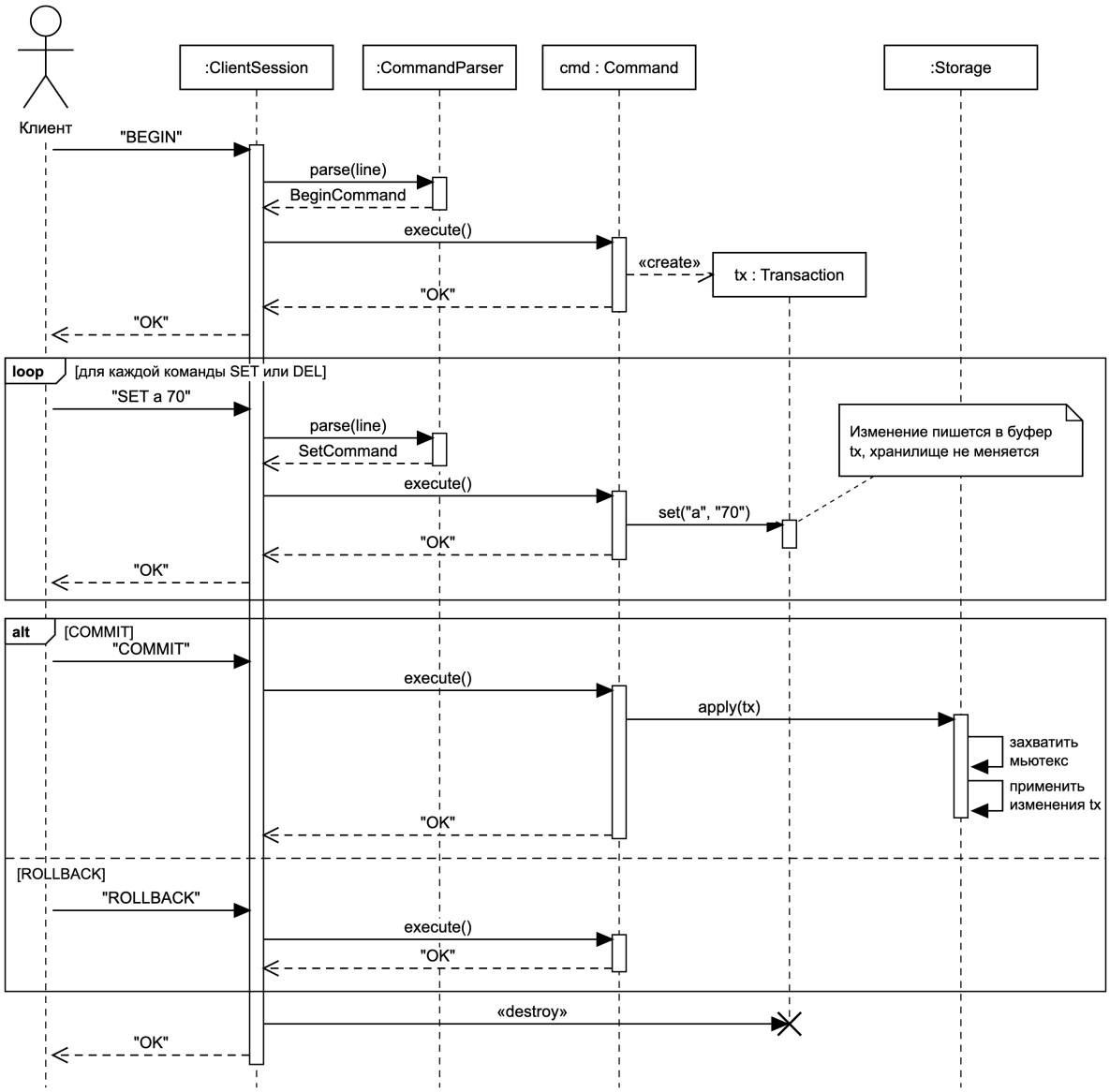


Рисунок 3 – Диаграмма последовательности «Выполнение транзакции»

Заключение

В ходе работы выбрана тема проекта, собраны требования (анализ документов и аналогов, интервью, пользовательские истории) и распределены по уровням К. Вигерса с приоритетами MoSCoW: бизнес-требований – 3, пользовательских – 9, функциональных – 21, нефункциональных – 10.

На основе требований составлена спецификация SRS и построены диаграммы вариантов использования, классов и последовательности. Следующий этап – реализация обязательных (Must) требований: сервера, команд SET, GET, DEL и транзакций.

Список использованных источников

1. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. – 3-е изд., доп. – М.: Русская редакция; СПб.: БХВ-Петербург, 2014. – 736 с.

2. IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications.

3. Документация Redis [Электронный ресурс]. – URL: https://redis.io/docs/latest/ (дата обращения: 20.10.2026).

4. Буч Г., Рамбо Дж., Якобсон И. Язык UML. Руководство пользователя. – 2-е изд. – М.: ДМК Пресс, 2006. – 496 с.